Análise e Desenvolvimento de Sistemas

Disciplina: CODING

Professora: Diego Abreu

Aluno: Giovani Amauri Frazão Brelaz

Matrícula: 04206364

Turno: Manhã

Aula 5 Parte 0 Introdução e Funções.docx

Atividade 1 - Do problema grande às pequenas responsabilidades

Considere o problema “Criar um sistema acadêmico”. Em vez de tratá-lo como uma única tarefa, decomponha-o em responsabilidades menores.

Sistema acadêmico -> cadastrar aluno -> calcular média -> verificar aprovação -> mostrar relatório

1. Escreva mais duas responsabilidades que poderiam existir nesse sistema.

2. Explique por que dividir o problema pode facilitar o desenvolvimento.

Ideia central: decompor é transformar um problema grande em problemas menores, cada um com uma responsabilidade mais clara.

1. Mais duas responsabilidades que poderiam existir:

   - Registrar frequência do aluno.
   - Matricular aluno na turma / disciplina.

2. Dividir facilita porque transforma um problema grande e confuso em vários problemas pequenos e claros, cada parte fica com uma única responsabilidade.

Atividade 2 - Nossa primeira função

Digite, compile e execute o programa abaixo.

```
#include <stdio.h>

void mostrarMensagem(void) {
printf("Ola!\n");

}

int main(void) {

    mostrarMensagem();

    return 0;

}
```

Identifique no código

· Nome da função criada: mostrarMensagem.

· Tipo de retorno: void, pois ela não devolve um valor.

· Parâmetros: nenhum, indicado por (void).

· Chamada: mostrarMensagem();

1. Nome da função criada:

MostrarMensagem

2. Tipo de retorno:

void - ela só executa uma ação (mostrar o texto na tela).

3. Parâmetros:

Nenhum. O (void) dentro dos parênteses indica que ela não recebe nada para funcionar.

4. Chamada da função:

mostrarMensagem(); - está dentro do main. É nessa hora que o programa pula lá para cima, executa o printf("Ola!\n"), e depois volta.

Atividade 3 - Reutilização: cabeçalho do sistema

Crie uma função chamada mostrarCabecalho() que imprima o bloco abaixo e chame-a três vezes em pontos diferentes do programa.

========== SISTEMA CODING ==========

A função deverá ter a forma:

void mostrarCabecalho(void) {

/* seus printf aqui */

}

Experimento

3. Execute primeiro repetindo os printf manualmente.

```c
#include <stdio.h>

int main(void) {

    printf("========== SISTEMA CODING ==========\n");
    printf("\n");
    printf("Menu Principal\n");

    printf("========== SISTEMA CODING ==========\n");
    printf("\n");
    printf("Cadastro de Aluno\n");

    printf("========== SISTEMA CODING ==========\n");
    printf("\n");
    printf("Relatorio Final\n");

    return 0;
}
```

```
==================== SISTEMA CODING ====================

Menu Principal
==================== SISTEMA CODING ====================

Cadastro de Aluno
==================== SISTEMA CODING ====================

Relatorio Final


...Program finished with exit code 0
Press ENTER to exit console.
```

4. Substitua as repetições por chamadas de mostrarCabecalho().

```c
#include <stdio.h>

void mostrarCabecalho(void) {
    printf("==================== SISTEMA CODING ====================\n");
    printf("\n");
}

int main(void) {
    mostrarCabecalho();
    printf("Menu Principal\n");

    mostrarCabecalho();
    printf("Cadastro de Aluno\n");

    mostrarCabecalho();
    printf("Relatorio Final\n");

    return 0;
}
```

```
input
==================== SISTEMA CODING ====================

Menu Principal
==================== SISTEMA CODING ====================

Cadastro de Aluno
==================== SISTEMA CODING ====================

Relatorio Final


...Program finished with exit code 0
Press ENTER to exit console.
```

5. Altere o texto do cabeçalho. Em qual versão a manutenção é mais simples?

```c
#include <stdio.h>

void mostrarCabecalho(void) {
    printf("==================== SISTEMA ACADEMICO ====================\n");
    printf("\n");
}

int main(void) {
    mostrarCabecalho();
    printf("Menu Configuracoes\n");

    mostrarCabecalho();
    printf("Cadastro de Professores\n");

    mostrarCabecalho();
    printf("Relatorio de Notas\n");

    return 0;
}
```

```
input
==================== SISTEMA ACADEMICO ====================

Menu Configuracoes
==================== SISTEMA ACADEMICO ====================

Cadastro de Professores
==================== SISTEMA ACADEMICO ====================

Relatorio de Notas


...Program finished with exit code 0
Press ENTER to exit console.
```

A manutenção é mais simples na versão com a função.

Se você precisar mudar o texto do cabeçalho, por exemplo de SISTEMA CODING para SISTEMA ACADEMICO, você muda só em 1 lugar, dentro da função mostrarCabecalho().

Na versão repetindo os printf manualmente, você teria que procurar e mudar em 3 lugares diferentes. Se esquecer de um, o sistema fica com cabeçalho diferente em cada tela.

Atividade 4 - Funções que já conhecíamos

Marque quais itens abaixo são funções em C e explique o papel de cada uma:

· printf()

· scanf()

· main()

· mostrarCabecalho()

Conexão: funções não são uma novidade absoluta: printf, scanf e main já estavam presentes desde as primeiras aulas. Agora você começa a definir suas próprias funções.

Todas são funções!

1. printf() - SIM, é função

Papel: Mostrar (imprimir) textos e valores na tela.

2. scanf() - SIM, é função

Papel: Ler o que o usuário digita no teclado e guardar numa variável.

3. main() - SIM, é função

Papel: É a função principal, onde todo programa em C começa.

4. mostrarCabecalho() - SIM, é função

Papel: Mostrar o cabeçalho do sistema.

Desafio - Menu reutilizável

Crie uma função void mostrarMenu(void) que apresente o menu abaixo. O main deve apenas chamar a função e encerrar.

1 - Somar 2 - Subtrair 0 – Sair



6. Qual é a diferença entre definir uma função e chamar uma função?

Resposta:

Definir: É criar a função, escrever o que ela vai fazer.
Ex: void mostrarMenu(void) { printf(...); }

Chamar: É usar a função, mandar ela executar.
Ex: mostrarMenu(); dentro do main.

Sem definir, não tem o que chamar. Sem chamar, a função existe mas nunca é executada.

7. Por que uma função chamada “fazTudo” tende a ser um sinal de má decomposição?

Resposta: Se uma função faz tudo, ela fica grande, confusa e difícil de testar e corrigir. Se der um erro em uma parte, você pode quebrar o resto.

8. Qual vantagem aparece quando o mesmo trecho precisa ser usado várias vezes?

Resposta: Você escreve o código uma única vez dentro da função e só chama ela quantas vezes precisar. Se precisar mudar o texto do menu, você muda em um só lugar e já muda em todo o programa, em vez de ter que procurar e mudar em vários lugares e correr o risco de esquecer algum.